

## Pseudocode

**Pseudocode** เป็นการอธิบายขั้นตอนการทำงานของโปรแกรมในรูปแบบลำดับขั้น

โดยใช้ข้อความที่เข้าใจง่ายและไม่ขึ้นอยู่กับไวยากรณ์ของภาษาโปรแกรม

เพื่อแสดงแนวทางการทำงานของอัลกอริทึมก่อนนำไปพัฒนาเป็นโปรแกรมภาษา Java ในโครงงานนี้

Pseudocode ครอบคลุมการทำงานของโปรแกรมหลัก อัลกอริทึม Brute Force อัลกอริทึม Monotonic Stack

การคำนวณ Largest Rectangle in Histogram และการทดลองวัดประสิทธิภาพของอัลกอริทึม

### 1 Pseudocode โปรแกรมหลัก (Main)

BEGIN

Create Scanner

Create NGEAlgorithm object

REPEAT

Display "Enter number of buildings"

IF input is not an integer THEN

Display "Invalid input"

Read invalid input

ELSE

Read number of buildings

END IF

IF number of buildings  $\leq 0$  THEN

Display "Number of buildings must be greater than 0"

END IF

UNTIL number of buildings  $> 0$

Create array heights with size n

FOR i = 0 TO n - 1

REPEAT

Display "Enter height of building i+1"

IF input is not an integer THEN

Display "Invalid input"

Read invalid input

ELSE

Read height

IF height  $< 0$  THEN

Display "Height must not be negative"

```

        END IF
    END IF
    UNTIL height is a valid non-negative integer
        Store height in heights[i]
    END FOR
    Display input data
    Start timer for Algorithm A
    resultA ← BruteForceNGE(heights)
    Stop timer
    Get operationA

    Start timer for Algorithm B
    resultB ← MonotonicStackNGE(heights)
    Stop timer
    Get operationB

    largestRectangle ← LargestRectangle(heights)

    Display resultA, timeA and operationA
    Display resultB, timeB and operationB
    Display largestRectangle

    IF resultA equals resultB THEN
        Display "Result comparison: PASS"
    ELSE
        Display "Result comparison: FAIL"
    END IF
    Close Scanner

END

```

หลักการทำงาน: โปรแกรมรับและตรวจสอบข้อมูลอาคาร จากนั้นประมวลผลด้วย Algorithm A และ Algorithm B เพื่อหา Next Greater Element พร้อมวัดเวลา จำนวน Operation เปรียบเทียบผลลัพธ์ และคำนวณ Largest Rectangle in Histogram

## 2 Pseudocode Algorithm A: Brute Force

```

ALGORITHM BruteForceNGE(heights)

```

```

operationA ← 0
n ← length of heights
Create result array of size n
FOR i = 0 TO n - 1
    result[i] ← -1
    FOR j = i + 1 TO n - 1
        operationA ← operationA + 1
        IF heights[j] > heights[i] THEN
            result[i] ← heights[j]
            BREAK
        END IF
    END FOR
END FOR
RETURN result
END ALGORITHM

```

หลักการ: ตรวจสอบอาคารทางด้านขวาทีละอาคารจนพบอาคารที่สูงกว่า หากไม่พบจะเก็บค่า -1

### 3 Pseudocode Algorithm B: Monotonic Stack

```

ALGORITHM MonotonicStackNGE(heights)

operationB ← 0
n ← length of heights
Create result array of size n
Create empty stack
FOR i = 0 TO n - 1
    result[i] ← -1
    WHILE stack is not empty
        operationB ← operationB + 1
        IF heights[i] > heights[stack.top()] THEN
            index ← stack.pop()
            operationB ← operationB + 1
            result[index] ← heights[i]
        ELSE
            BREAK
    END WHILE
END FOR

```

```
        END IF
    END WHILE
    stack.push(i)
    operationB  $\leftarrow$  operationB + 1
END FOR
RETURN result
END ALGORITHM
```

#### 4.Pseudocode การทดลองประสิทธิภาพ

สำหรับ Experiment.java ที่ใช้ทดสอบขนาดข้อมูล 10, 100, 1000, 5000 และ 10000

```
BEGIN Experiment

    Define data sizes:
        10, 100, 1000, 5000, 10000
    FOR each data size n
        Generate test data with size n
        Run Algorithm A
        Measure execution time
        Count operations
        Run Algorithm B
        Measure execution time
        Count operations
        Compare results from Algorithm A and Algorithm B
        IF results are equal THEN
            Display "PASS"
        ELSE
            Display "FAIL"
        END IF
        Display time and operations
        for both algorithms
    END FOR
```

Display "Experiment completed"

END